

Linea y circunferencia de Bresenhamn

Graficación por Computadora

Mauricio Rodriguez Gaytan

Grupo: 1059

Profesora: Liliana Hernández Cervantes

Función “graficar”

```
#Esta funcion se va a encontrar de graficar los puntos
def graficar(titulo, puntos): 2+ usages
    #Con esto se extrae todas las coordenada 'x' y las coordenadas 'y'
    #y las guarda en listas separadas
    x = [p[0] for p in puntos]
    y = [p[1] for p in puntos]

    plt.plot(*args: x, y, 'bs') # 'bs' dibuja los cuadros azules
    plt.grid(True) # Activa la cuadrícula que aparece en el fondo
    plt.title(titulo) #Muestra el titulo en la pantalla
    plt.axis('equal') # Hace que la pantalla tenga la escala correcta
    plt.show() #Renderizar la imagen donde se mostrara la grafica
```

Para qué sirve: Se encarga de renderizar el programa, recibe las listas de coordenadas después de para traducirlas a una representación visual en pantalla.

Una vez que tiene las listas, separa las tuplas de puntos calculados en dos arreglos individuales, uno para las coordenadas X y otro para las Y.

Renderización: Utiliza el argumento 'bs' de Matplotlib para dibujar cuadros azules en lugar de puntos para poder mostrar cómo se verían los píxeles renderizados

Función “bresenham_linea”

```
def bresenham_linea(x1:(_t_), y1:(_t_), x2:(_sub_), y2:(_sub_)):
    print(f"\n-- LÍNEA ({x1},{y1}) a ({x2},{y2}) ---")
    #Distancias absolutas sin signo entre los puntos iniciales y finales
    dx, dy = abs(x2 - x1), abs(y2 - y1)
    #Determinan la dirección de dibujo, con esto se hacen las reglas
    #1 si avanza hacia la derecha y arriba
    #-1 si retrocede
    sx = 1 if x1 < x2 else -1
    sy = 1 if y1 < y2 else -1
    #steep revisa si la línea crece más en Y que en X en un ángulo mayor a 45°
    #si si es mayor se intercambia dx y dy temporalmente para que el algoritmo
    #siempre avance sobre el eje dominante
    steep = dy > dx
    if steep:
        dx, dy = dy, dx
    #parametro de decision inicial P0 = 2dy - Δx
    pk = 2 * dy - dx
    #aquí se toma el valor del punto de inicio
    x, y = x1, y1
    #aquí se guardan los pixeles
    puntos = [(x, y)]
```

```
#Este ciclo se repite dependiendo de los pixes dominantes que haya en el eje dominante (dx)
for _ in range(dx):
    #guardar el pk antes de modificarlo
    curr_pk = pk
    #si el pl < 0 la línea ideal está cerca, por lo que no se avanza en
    #el eje secundario
    if pk < 0:
        pk += 2 * dy #actualizar el pk

        if steep:
            y += sy
        else:
            x += sx
    else:
        pk += 2 * (dy - dx)
        x += sx
        y += sy
    print(f"Pk: {curr_pk}>4} -> (x, y): ({x}, {y})")
    puntos.append((x, y))
graficar( titulo: f"Línea ({x1},{y1}) a ({x2},{y2})", puntos)
```

Función “bresenham_linea”

Para qué sirve: Se encarga de resolver la secuencia óptima de píxeles para representar la línea entre los puntos dados, utilizando cálculos de numeros enteros (sumas, restas y multiplicaciones por 2).

Evalúa el parámetro (P_k): A partir del valor inicial P_0 , el algoritmo decide en cada paso si el píxel se mantiene en la misma coordenada secundaria $P_k < 0$ o si debe aumentar ambas coordenadas para seguir la trayectoria ideal de la recta.

Registro: Imprime en consola cada cambio de forma antes de actualizar las variables, replicando exactamente el comportamiento de una prueba de escritorio tradicional, con esto genera la tabla de cálculos que se muestra en la terminal.

Función “bresenham_circulo”

```
def bresenham_circulo(r):
    print(f"\n--- CÍRCULO r={r} ---")
    x, y, pk = 0, r, 1 - r
    puntos = []
    k = 0
    while x < y:
        curr_pk = pk
        x += 1
        if pk < 0:
            pk += (2 * x) + 1
        else:
            y -= 1
            pk += (2 * x) + 1 - (2 * y)
        print(f"k: {k:>2} | Pk: {curr_pk:>4} | ({x}, {y}) | 2xk: {2 * x:>2} | 2yk: {2 * y:>2}")
        # Agregar los 8 puntos simétricos en una sola instrucción
        simetricos = [(x, y), (-x, y), (x, -y), (-x, -y), (y, x), (-y, x), (y, -x), (-y, -x)]
        puntos.extend(simetricos)
        k += 1
    graficar( titulo: f"Círculo r={r}", puntos)
```

Para qué sirve: Se encarga de dibujar la circunferencia sin utilizar funciones trigonométricas, el ciclo while hace las operaciones matemáticas sólo para un segmento de 45 grados de la circunferencia, deteniendo el cálculo en el momento exacto en que la coordenada X alcanza a la coordenada Y.

Simetría: Por cada píxel individual calculado, el algoritmo deduce sus 7 píxeles correspondientes ("espejos") simplemente intercambiando signos y el orden de (x, y), junta al mismo tiempo las 8 coordenadas en cada ciclo del bucle, para formar el círculo completo y enviarlo a la parte gráfica para dibujarlo.

Ejecución de ejemplos

```
#Ejemplo de ejecucion
```

```
#bresenham_linea(10, 12, 35, 26) # Ejercicio 1  
# bresenham_linea(17, 21, 35, 42) # Ejercicio 2  
# bresenham_linea(2, 5, 7, 18)    # Ejercicio 3  
# bresenham_linea(20, 10, 32, 18) # Ejercicio 4  
# bresenham_circulo(20)           # Ejercicio Circulo
```

Para qué sirve: Se llama a la función con los valores exactos necesita en los ejercicios.

Control de ejecución: Utiliza el carácter de comentario para habilitar o deshabilitar líneas de código específicas y con esto ejecutar solo caso a la vez

Fuentes de información

- GeeksforGeeks. (2025, 23 julio). *Bresenham's Line Generation Algorithm*. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/bresenhams-line-generation-algorithm/>
- HuluWZ. (2021). *Line-And-Circle-Drawing-Algorithms*. Github. Recuperado 8 de octubre de 2026, de
<https://github.com/HuluWZ/Line-And-Circle-Drawing-Algorithms>